

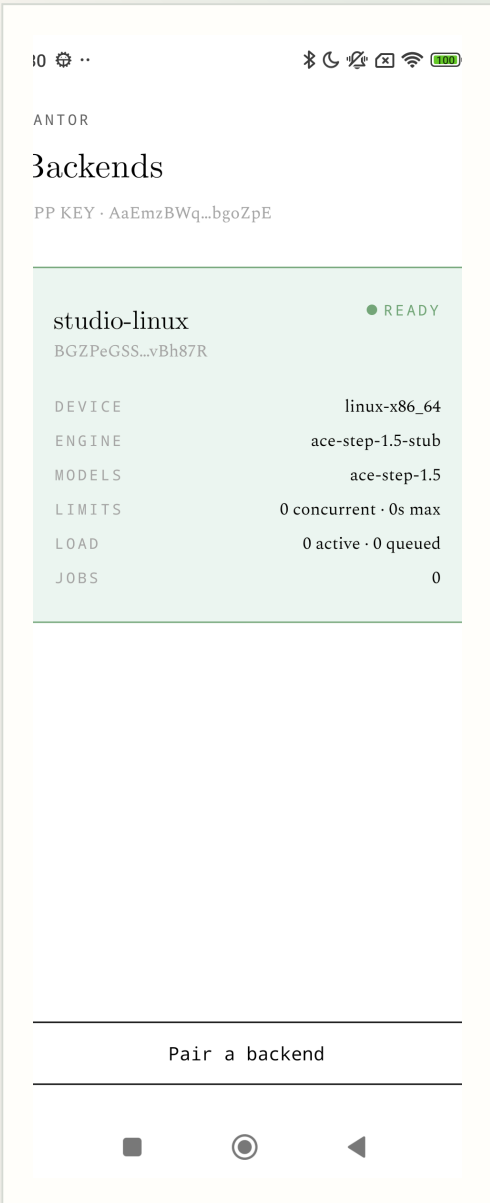
THE FIRST CONNECTION IS REAL

A private phone-to-compute path, deployed end to end.

Cantor can now discover, pair with, authenticate, and monitor a Linux backend through a production Cloudflare relay—without opening a home port and without teaching the relay the one-time pairing secret.

● LIVE PROTOTYPE · PRODUCTION RELAY

What this proves: the identity, relay, pairing, protocol, reconnect, status, and deployment seams that generation will use are working on a physical Android device. Generation itself remains deliberately out of scope for this milestone.



READY

Physical Android app against the redeployed production room

143

Automated checks: 120 app + 18 Rust + 5 Worker

100%

Production traffic on Worker version db43626e...

0

Inbound ports or generation jobs required for this proof

What exists today

The milestone is intentionally a thin vertical slice: every trust and transport boundary is real, while GPU generation remains stubbed.

ANDROID APP

Mnemonic-derived Ed25519 identity, Keystore protection, QR pairing, persisted backend, reconnect and status UI.

BLIND RELAY

Cloudflare Worker + one hibernating Durable Object per node key. Presence and opaque splice only.

LINUX NODE

Stable Ed25519 room identity, signed room claim, allowlist, one-time enrollment, reconnect and installer stub.

Trust boundaries

- The node proves room ownership with an Ed25519 signature.
- The app proves possession of its own key during the node handshake.
- The QR token stays off the relay-visible wire; enrollment uses a domain-separated, key-bound HMAC proof.
- Transient Keychain read failures cannot fall into identity creation.

Operational behavior

- The newest valid node claim owns the room.
- Presence changes propagate to attached clients.
- Both app and node reconnect with capped jittered backoff.
- Detached clients release node session state; active sessions are capped at 1,024.

Production proof

- `wss://cantor.ckadirt.xyz` is live with a valid TLS certificate.
- Hardened Worker version: `db43626e-1ab1-4382-8f4b-85dcc643d861`.
- The updated `studio-linux` node reclaimed its production room.
- The stored phone identity reconnected and rendered **READY**.

Deliberately not claimed

- No song generation or engine integration yet.
- No large audio transfer or generation data plane yet.
- No literal LTE acceptance test: the available phone has no SIM.
- Installer release assets are not yet published.

Why this is a useful prototype: it retires the riskiest architectural uncertainty before generation work begins. The next phase can focus on jobs, progress, artifacts, and engine execution without reopening identity, pairing, presence, or room ownership.

DECISION

The buy-in ask

Approve the first-connection foundation as the baseline and move generation onto these established seams.

RECOMMENDED DECISION

Green-light the generation phase.

Keep the deployed identity and relay architecture stable. Add generation as versioned application messages and durable jobs, following the locked phase plan and using the older protocol guide only where it does not conflict.

- Define the minimal generation request/progress/artifact slice.
- Seal sensitive payloads before lyrics or audio cross the relay.
- Publish signed/checksummed Linux node release assets.
- Run a true cross-network acceptance test when a second internet path is available.

Known risks are bounded

VERSION SKEW	Rust is the protocol source and committed TypeScript bindings are generated in tests.
RELAY TRUST	The relay routes by room/session metadata but does not own node or client identity keys.
RESOURCE USE	Reconnects are capped and jittered; client session state now has detach cleanup and a hard ceiling.
SCOPE CREEP	The locked plan wins over earlier guides. Generation-only records stay deferred until that phase.

01	Locked build plan	Authoritative first-connection decisions and phase boundaries.
02	Backend protocol draft	Supporting detail for the longer-term versioned job protocol.
03	Network & identity explainer	Supporting rationale for relay, WebSocket, identity, and trust choices.
04	Implementation notes	Complete record of Phases 1–4, hardening, validation, and deviations.

Reading rule: the locked final plan is authoritative wherever documents differ. Earlier guides supply referenced detail only when compatible. The implementation notes record conservative deviations and the evidence actually observed.

The first connection

A concrete plan for the first real piece of network infrastructure: the relay on Cloudflare, a stub Linux node, and the app pairing with it — the green dot, end to end, with every seam in place for real generation later. All eight questions from draft 1 are answered, folded into the plan, and marked in green where they changed something. Next stop is Phase 0.

1 · The milestone, in one paragraph

You open Cantor on your phone. In settings there's a new screen: *Backends*. On your Linux machine you run one command; it prints a QR code. You scan it. A card appears on the phone — "**cesar-desktop**", with a green dot — even though the phone is on LTE and the laptop is behind your home router. Tap the card and you see what that backend says about itself: device type, engine version, the models it claims to have, its limits. Kill the laptop's Wi-Fi and the dot goes gray within seconds. Restore it, green again. **No generation happens.** But the entire pipeline that generation will ride on — identity keys, relay, presence, pairing, the versioned protocol, reconnect logic — is real and deployed.

Why this milestone first: it forces every hard seam (keys, relay, protocol envelope, reconnect) while the payload is trivially simple. When the GGML engine is ready, generation becomes "add three message types," not "build a network."

2 · The three pieces



both connections point INWARD — nobody needs a public IP

DECIDED · ONE REPO, THREE FOLDERS

Everything lives in the existing repo alongside `cantor/`. The deciding factor is types: `protocol/` is generated from Rust and imported by the app, so one commit changes a message shape everywhere and version skew is structurally impossible. If the relay or node ever grows its own contributors, splitting later is a `git filter-repo` away — but we don't pay that cost now.

FOLDER	LANGUAGE	WHAT IT DOES IN THIS MILESTONE
<code>relay/</code>	TypeScript	Cloudflare Worker + one Durable Object class. Accepts WebSockets from nodes and phones, verifies a node owns its pubkey, splices frames between them, broadcasts presence. Never parses the payload.
<code>node/</code>	Rust	Small binary. Loads or mints a keypair, keeps one outbound WebSocket to the relay, answers the handshake, serves a static <code>NodeInfo</code> and an empty jobs list. The engine slot is a stub.
<code>protocol/</code>	TS (generated)	Emitted by <code>cargo test</code> in <code>cantor-proto</code> via ts-rs. Committed, never hand-edited. The app imports from here.
<code>cantor/</code>	TS / RN	Derives the user's Ed25519 identity from the onboarding words, adds a pairing screen + backend picker, maintains the relay connection with reconnect, renders presence and <code>NodeInfo</code> .

3 · The relay: Worker + Durable Object

This is the Cloudflare piece, and it's deliberately small — under ~400 lines. The design follows one rule from the architecture: **the relay moves envelopes; it never opens them.**

One Durable Object per node

Every backend node gets its own Durable Object instance, addressed by the node's public key: `env.R00MS.getByName(nodePubkey)`. Cloudflare guarantees the same name always reaches the

same object, wherever it lives on the edge. So a "room" is: one node's socket, plus the sockets of every phone currently watching that node. This is exactly the coordination-atom shape Durable Objects are built for — no global state, no cross-room traffic, scales by doing nothing.

The room uses the **WebSocket Hibernation API** (`ctx.acceptWebSocket()` , not `ws.accept()`): when nobody is talking, the object is evicted from memory while Cloudflare's edge keeps the TCP connections alive and answers pings automatically (`setWebSocketAutoResponse`). Billable duration during idle is zero — this is what makes the free tier genuinely free for a relay whose job is mostly holding open lines. Per-socket state (role, authed, session id) is stored with `serializeAttachment()` so it survives hibernation.

Connecting and claiming a room

Both sides connect to the same URL shape:

```
GET wss://cantor.ckadirt.xyz/v1/room/<node-pubkey>?role=node
GET wss://cantor.ckadirt.xyz/v1/room/<node-pubkey>?role=client
```

A *client* (phone) may attach to any room freely — knowing a pubkey only lets you knock; the real lock is the end-to-end handshake with the node itself. A *node*, however, must prove it owns the pubkey before the relay will treat it as the room's node — otherwise a stranger could squat your room and eat your messages. That proof is a three-frame relay-level handshake:

```
relay → node { v:1, t:"relay.challenge", nonce }
node → relay { v:1, t:"relay.claim", pubkey, sig } // sig = sign(nonce)
relay → node { v:1, t:"relay.ok" } // socket is now THE node socket
```

Workers' WebCrypto supports Ed25519 natively, so verification is a few lines with no dependencies. A new claimed node socket replaces the old one (reconnect-wins semantics).

The relay's whole vocabulary

Six frame types. Everything else that will ever exist — hello, auth, submit, job progress, audio chunks — travels inside the opaque `payload` and the relay never learns the difference:

FRAME	DIRECTION	MEANING
<code>relay.challenge / relay.claim / relay.ok</code>	relay ↔ node	Prove pubkey ownership, claim the room.
<code>relay.presence { online }</code>	relay → clients	Pushed on attach and whenever the node's socket opens or closes. This is

the green dot.

```
tunnel { sid?, payload }
```

both ways

Client → relay: forwarded to the node as `tunnel { sid, payload }`, where `sid` is a relay-assigned ephemeral session id. Node → relay with a `sid`: routed to that client. The relay learns which *sessions* talk, never which *identities* — client pubkeys only ever appear inside the payload.

```
relay.error { code, msg }
```

relay →
either

e.g. `node-offline`, `bad-claim`.

Project shape

```
relay/  
  wrangler.jsonc      // name: cantor-relay, D0 binding ROOMS → NodeRoom  
  src/  
    index.ts          // Worker: route /v1/room/:pubkey → getByName(pubkey)  
    room.ts           // NodeRoom D0: claim, presence, splice — the whole relay  
    frames.ts         // relay frame types (hand-written TS; tiny, relay-only)  
  test/  
    room.test.ts      // @cloudflare/vitest-pool-workers
```

```
// wrangler.jsonc (essentials)  
{  
  "name": "cantor-relay",  
  "main": "src/index.ts",  
  "compatibility_date": "2026-07-01",  
  "durable_objects": { "bindings": [{ "name": "ROOMS", "class_name": "NodeRoom" }] },  
  "migrations": [{ "tag": "v1", "new_sqlite_classes": ["NodeRoom"] }],  
  "routes": [{ "pattern": "cantor.ckadirt.xyz", "custom_domain": true }]  
}
```

SQLite-backed from day one even though this milestone barely stores anything — the mailbox (store-and-forward for offline peers) and push-token registry land in this same object later, and starting on `new_sqlite_classes` means no migration pain then.

DECIDED • DOMAIN

cantor.ckadirt.xyz now, since that zone is already on your Cloudflare account — a custom domain on a Worker is one dashboard click (or the `routes` entry above) and gets its cert automatically. **cantor.fm** when you buy it: add a second route, keep the old one alive, and paired backends keep working because the relay URL is stored per-backend as data. Worth

noting for later — that's exactly the moment to make sure the app can be told a *new* relay URL for an existing pairing.

DECIDED · ENCRYPTION COMES LATER, WITH FOCUS

Payloads stay plaintext-inside-a-blind-pipe for this milestone. The relay code never parses `payload`, so the structural seam is already correct; sealing changes only what's inside it and needs **zero relay changes**. We'll give it its own dedicated pass before real lyrics or audio ever cross the wire.

Free-tier reality check

Workers free plan: ~100k requests/day; Durable Objects included with SQLite storage (1 GB). Incoming WebSocket messages are metered cheaply (batched ~20:1 against the request quota), hibernation zeroes idle duration, and auto ping/pong doesn't even wake the object. One person with a phone and two nodes exchanging handshakes and presence is deep, deep inside free. Self-hosting story stays what we designed: a friend runs `wrangler deploy` on their own Cloudflare account and points the app at their URL.

4 · The node: a small Rust binary

DECIDED · RUST FROM DAY ONE

No throwaway script. The stub node is ~300–400 lines, and it's the crate that will eventually hold the engine FFI and do the actual compute — the efficiency matters when this box is running a diffusion model, and `ts-rs` living in Rust is what keeps the app's types honest. The head start is real infrastructure, not ceremony.

The workspace starts with the crate layout from the protocol draft, with the engine crate simply absent:

```
node/
Cargo.toml           // workspace
crates/
  cantor-proto/       // serde structs + ts-rs → generates ../protocol/*.ts
  cantor-node/        // the binary: relay line, handshake, stub NodeInfo
install.sh            // curl | sh stub: fetch binary, write config, systemd unit
```


`cantor-engine` (the FFI wrapper over `cantor_engine.h`) appears when the GGML pipeline is ready. For now `cantor-node` answers `describe` with a hardcoded `NodeInfo` ("ace-step-1.5, stub") and `status` with an empty jobs list.

What the binary does

1. **First run:** mint an Ed25519 keypair → `~/.config/cantor/node.key` (mode 0600); write `node.toml` (name, relay_url, allowed_keys).
2. **cantor-node pair:** print the pairing QR in the terminal (via the `qrcode` crate) plus the same thing as a copyable string: `cantor://pair?pk=<base58-pubkey>&relay=<url>&name=cesar-desktop`.
3. **cantor-node run:** connect out to the relay (`tokio-tungstenite`), answer `relay.challenge`, then loop: for each `tunnel` frame, run the end-to-end handshake (hello → challenge → auth, checking the client's pubkey against `allowed_keys`) and answer `describe` / `status`.
Reconnect with jittered exponential backoff, forever.

Crates: `tokio`, `tokio-tungstenite`, `ed25519-dalek`, `serde` / `serde_json`, `ts-rs`, `qrcode`, `toml`. No axum yet — no inbound listener in this milestone, relay line only; the direct-LAN listener is a later phase that slots into the same message loop.

NOTED FOR LATER · GENERATION IS MULTI-STEP, AND STATUS IS CONSULTABLE

Your point about staged generation — lyrics first, then the DiT/diffusion pass, then decode — is **not built now**, but it's the reason two things in this milestone are shaped the way they are, and I want them on the record before we write code:

1. Status is a question, not just a broadcast. The node answers `status` with the full current state on demand, not only pushing events. That's why the phone can reopen after being closed for an hour and immediately know "step 34 of 57, diffusing" — it *asks*. We're implementing that query in this milestone even though the answer is an empty list, precisely so the habit exists everywhere before it matters.

2. A job is a sequence of stages, not one opaque wait. `JobView` already carries `stage` alongside `step / steps_total`, so a job can legitimately report `stage:"lyrics"` then `stage:"diffuse" 34/57` then `stage:"decode"`, each with its own progress. When we design generation properly, the open questions will be whether stages become independently *addressable* — submit lyrics only, review them, then submit the diffusion pass against a stored lyrics artifact — and whether `interrupted` resumes at stage granularity. Both are plan-later; neither is blocked by anything we're building now.

5 • The app: identity, pairing, presence

Identity: derive the keys from the words you already generate

DECIDED • THE MNEMONIC IS THE ROOT, AND THE KEYS STAY INVISIBLE

The onboarding already generates 12 words, so we don't mint a separate keypair — we **derive** the Ed25519 identity from that mnemonic. This is strictly better than draft 1: the words become a genuine backup rather than decoration, and restoring them on a new phone reproduces the *same public key*, so every backend you ever paired still recognizes you with no re-pairing. The keys themselves are never shown — they're long, unreadable, and (as you said) not the thing a person should be asked to care about. The words are the visible artifact; the keys are plumbing generated silently behind them.

The derivation, precisely — deterministic, standard, and domain-separated so the same phrase can safely produce other keys later:

```
words  - bip39.mnemonicToSeedSync(phrase)          // @scure/bip39 → 64-byte seed
        - hkdf(sha256, seed, info:"cantor-identity-v1") // @noble/hashes → 32 bytes
        - ed25519 secret key → public key = your identity
```

The derived secret key goes into Android Keystore via `react-native-keychain`; the mnemonic itself is only ever held long enough to derive (and shown once during onboarding, as it already is). Sign/verify with `@noble/ed25519` — pure TS, audited, no native build step, trivially testable in Jest, and we sign a few bytes a few times per connection so performance is irrelevant.

One detail worth flagging: 12 words is 128 bits of entropy, which is entirely fine for this. Earlier sketches said "16 words" — 12 is the standard and I'd keep whatever the onboarding already produces rather than churn the UX.

Backends screen + pairing

DECIDED • QR SCANNING, NO TYPEABLE CODE

Since there's no camera dependency yet, we add `react-native-vision-camera` — the current standard, with code scanning built in, no separate barcode library. QR is the primary pairing flow, with the `cantor://pair?...` string pasteable as the fallback for SSH-only boxes.

The short typeable temporal code is dropped from this milestone; it needs the relay to hold short-lived pairing state, which is complexity we don't need to prove anything here.

- **Backend picker:** a list of paired backends — petname, device glyph, green/gray dot, "last seen". Local storage per backend: `{ nodePubkey, relayUrl, petname, lastNodeInfo }`.
- **Pairing:** scan QR (or paste the string) → parse → connect → handshake → store → card appears.
- **Backend detail:** renders `NodeInfo` verbatim — device type, engine version, models, limits, current load. Nothing hardcoded; the card is a projection of what the node said. This is the "NodeInfo drives the UI" seam being exercised for the first time.

The connection store

One module owns each relay socket and exposes plain state to the UI (the UI renders state, never events — same resilience rule as jobs):

```
disconnected → connecting → attached      // on relay, node offline (gray dot)
                  → handshaking           // node online, e2e hello/auth in flight
                  → ready                  // green dot, NodeInfo fresh
```

Reconnect on drop with jittered backoff; on every reconnect, re-run the handshake and re-fetch `status` — the resync habit, installed before there are even jobs to resync. Kill the app, reopen: it reconnects and the dots settle to truth within a few seconds.

Type generation

`cargo test` in `cantor-proto` emits `protocol/*.ts`; the app imports those files directly. Rust is the source of truth from the first message. The relay's six frames stay hand-written TS — the relay is intentionally not a protocol participant.

6 • The wire, end to end

The complete conversation this milestone implements — relay frames on the outside, protocol-vo envelopes (from the [protocol draft](#)) on the inside:

```
phone → relay  tunnel { payload: { v:1, id, t:"hello", pubkey } }
relay → node   tunnel { sid:"s1", payload: ...hello }
node  → relay  tunnel { sid:"s1", payload: { v:1, id, t:"challenge", nonce, node_pubkey } }
```

```

phone          verifies node_pubkey == the one it paired with (anti-impostor)
phone → ...    tunnel { payload: { v:1, id, t:"auth", sig } }
node           checks pubkey ∈ allowed_keys, verifies sig
node → ...    tunnel { sid:"s1", payload: { v:1, id, t:"welcome", node: NodeInfo } }
phone → ...    tunnel { payload: { v:1, id, t:"status" } }
node → ...    tunnel { sid:"s1", payload: { v:1, id, t:"jobs", jobs: [] } } // empty, and that

```

Everything future — `submit`, staged `job` progress pushes, binary audio chunks — is new values of `t` inside the same envelopes over the same two hops. That's the whole point of building this skeleton first.

7 • Build order

Five phases, each ending in something you can see working. Rough effort in focused sessions, not calendar time.

PHASE 0 • ~1 session **Scaffold + local loop**

Create `relay/` (wrangler + DO skeleton) and `node/` (workspace, keygen, config). `wrangler dev` runs the relay locally; the Rust node connects to `ws://localhost:8787` and claims its room.

✓ demo: terminal shows `relay.ok – room claimed as cesar-desktop`

PHASE 1 • ~1–2 sessions **Presence + splice, tested**

Full relay vocabulary: claim verification (WebCrypto Ed25519), presence broadcast, tunnel splicing with session ids, reconnect-wins, hibernation + auto ping/pong. Vitest coverage for the room (claim, presence transitions, splice, bad-claim). A tiny `wscat`-style script plays the client role.

✓ demo: two terminals: kill the node process, watch the fake client print `presence: offline`, restart, `online`

PHASE 2 • ~1–2 sessions **Protocol crate + node handshake**

`cantor-proto` with the milestone's message set (hello/challenge/auth/welcome/status/jobs/error + NodeInfo + envelope), ts-rs emitting `protocol/*.ts`. Node implements the e2e handshake, allowlist check, static NodeInfo, `pair` subcommand with QR.

✓ demo: script client completes hello→auth→welcome and prints the NodeInfo JSON; a non-allowlisted key gets a clean `rejected`

PHASE 3 • ~2–3 sessions **The app joins**

Key derivation from the onboarding mnemonic + keystore storage, Backends screen, QR pairing with vision-camera, connection store with reconnect/resync, NodeInfo card. Points at the *local* wrangler dev relay over LAN.

✓ demo: the green dot on a real phone; toggle the node's process and watch it flip

PHASE 4 • ~1 session **Deploy + real-world test**

`wrangler deploy` with the custom domain on `cantor.ckadirt.xyz`. `install.sh` stub for the node (binary + config + systemd unit). The real test: phone on LTE, node on home Wi-Fi, nothing port-forwarded anywhere.

✓ demo: the milestone paragraph from section 1, exactly as written

Explicitly out of scope (designed-for, not built)

- Generation and jobs — including the staged/consultable design noted in section 4; rides the same tunnel.
- Payload sealing, mailbox store-and-forward, FCM wake pings — each a later phase, each with its seam already in the frame design.
- Direct-LAN / direct-VPS connections, hole punching, Iroh — the connection store's transport is already an interface; these are new implementations of it.
- Typeable pairing codes, profiles, petname sync, public sharing index, `cantor.fm` cutover.



SPEC FOR DISCUSSION

One conversation, every backend

The messages a Cantor node exposes — identical whether the engine runs inside your phone, on your laptop, on a VPS, or on our future cloud — plus the architecture of the Rust server that speaks it.

PART 1 · PRINCIPLES

Six rules before any message exists

1 · ONE PROTOCOL The app speaks exactly one language. Only the pipe changes: in-process (JSI), LAN, or relay. If a feature can't work over all three, it's designed wrong.	2 · STATE, NOT EVENTS Events are a convenience; state is the truth. Any client can miss any event and fully recover by asking. Reconnect-and-ask is the normal path, not the error path.	3 · CLIENT NAMES THE JOB Job IDs are minted by the phone (ULIDs). Resubmitting after a dead socket is harmless — the node says "already have it." Idempotency by construction.
4 · THE NODE DESCRIBES ITSELF Parameter ranges, model list, limits, queue length — the node tells the app on connect. The app hardcodes nothing; a bigger server just advertises bigger numbers.	5 · VERSIONED ENVELOPE Every message carries <code>v</code>. Nodes advertise the range they speak; old apps and new nodes negotiate instead of breaking.	6 · TEXT CONTROLS, BINARY CARRIES Control messages are JSON text frames — debuggable with your eyes. Bulk audio moves as binary frames with a tiny header. Never base64, never JSON-wrapped audio.

PART 2 · A SESSION

The shape of one conversation

1 **Connect**

The transport delivers an encrypted pipe — JSI (nothing to encrypt), direct WSS, or a relay tunnel. The protocol above it neither knows nor cares which.

2 **Prove who you are — both ways**

`hello` → `challenge` → `auth`. The client signs the node's nonce; the node signs the client's. Client checks the node key it pinned at pairing; node checks its allowlist. In-process transport skips this — you are already inside your own phone.

3 **The node introduces itself**

`welcome` carries the full `NodeInfo`: models, limits, parameter ranges, current queue. The backend picker UI is rendered from this, nothing else.

4 **Resync**

First real message, every time: `status {}` → `jobs [...]` — everything this node knows about your jobs. The UI redraws from it. This is the reconnect story from the network doc, made concrete.

5 **Work**

`submit` → `accepted`, then `job` events as stages advance. `cancel` at will. When a job is `done`, `fetch` pulls the audio as resumable binary chunks.



PART 3 • THE TYPES

Message catalog

Shown as TypeScript for readability — but see Part 6: these live in Rust and the TypeScript is *generated*. Every frame shares one envelope:

```
// every JSON frame, both directions
type Envelope = {
  v: 1 // protocol version
  id: string // echoes the request id in replies; fresh for pushes
  t: string // message type, below
  ...payload
}
```

Client → node

TYPE	PAYLOAD	REPLY
hello	<code>pubkey</code> , <code>nonce</code> , <code>proto: {min, max}</code>	challenge
auth	<code>sig</code> — signature over the node's nonce	welcome · error
status	<code>jobs?: JobId[]</code> — omit for "all mine"	jobs
submit	<code>job: JobId</code> (client-minted ULID), <code>params: GenParams</code>	accepted · rejected
cancel	<code>job: JobId</code>	job
fetch	<code>job: JobId</code> , <code>offset: number</code> — resume point in bytes	result + binary frames
describe	— (refresh NodeInfo mid-session)	welcome
allow.add	<code>pubkey</code> , <code>note?</code> — owner only	allowed
allow.remove	<code>pubkey</code> — owner only	allowed
allow.list	— owner only	allowed

Node → client

TYPE	PAYLOAD	WHEN
challenge	<code>nonce</code> , <code>sig</code> — node signs the client's nonce	after <code>hello</code>

TYPE	PAYLOAD	WHEN
welcome	<code>node: NodeInfo</code>	after <code>auth</code> , or <code>describe</code>
jobs	<code>jobs: JobView[]</code>	reply to <code>status</code>
accepted	<code>job</code> , <code>position</code> — or the existing <code>JobView</code> if resubmitted	reply to <code>submit</code>
rejected	<code>job</code> , <code>code</code> , <code>msg</code> — bad params, queue full, unknown model	reply to <code>submit</code>
job	<code>job: JobView</code> — full view, never a delta	pushed on any change
result	<code>job</code> , <code>meta: ResultMeta</code> , then binary frames	reply to <code>fetch</code>
allowed	<code>entries: {pubkey, note, role}[]</code>	reply to <code>allow.*</code>
error	<code>code</code> , <code>msg</code> , <code>retryable: bool</code>	anything malformed

The core records

```

type GenParams = {
  prompt: string           // style / tags / description
  lyrics?: string          // omit → instrumental
  duration_s: number       // clamped to NodeInfo.limits
  seed?: number            // omit → node picks; ALWAYS echoed back resolved
  steps?: number           // diffusion steps; default from node
  guidance?: number        // cfg scale; default from node
  model: string             // one of NodeInfo.models[].id
  format?: 'flac' | 'opus' // master vs small; default flac
}

type JobView = {
  job: JobId                // the client's ULID
  state: 'queued' | 'running' | 'interrupted'
    | 'done' | 'failed' | 'cancelled'
  stage?: 'encode' | 'diffuse' | 'decode' // while running
  step?: number             // diffuse only: 34

```

```

steps_total?: number           // diffuse only: 57
eta_s?: number                 // node's honest guess
position?: number              // while queued
error?: { code: string, msg: string }
params: GenParams               // echoed, seed resolved – full reproducibility
created_at: string             // node clock, ISO-8601 UTC
}

type NodeInfo = {
  name: string                  // "Cesar's Hetzner box" – self-asserted
  pubkey: string
  proto: { min: 1, max: 1 }
  engine: { version: string, commit: string }
  device: 'phone' | 'desktop' | 'server'
  models: { id: string, name: string, checkpoint_sha: string }[]
  limits: {
    max_duration_s: number,
    steps: { min: number, max: number, default: number },
    guidance: { min: number, max: number, default: number },
    queue_max: number, concurrent: number
  }
  load: { queued: number, running: number }
  retention_days: number       // how long results survive on the node
}

type ResultMeta = {
  job: JobId
  bytes: number
  sha256: string               // verify after resumable download
  format: 'flac' | 'opus'
  sample_rate: number
  duration_s: number
}

```

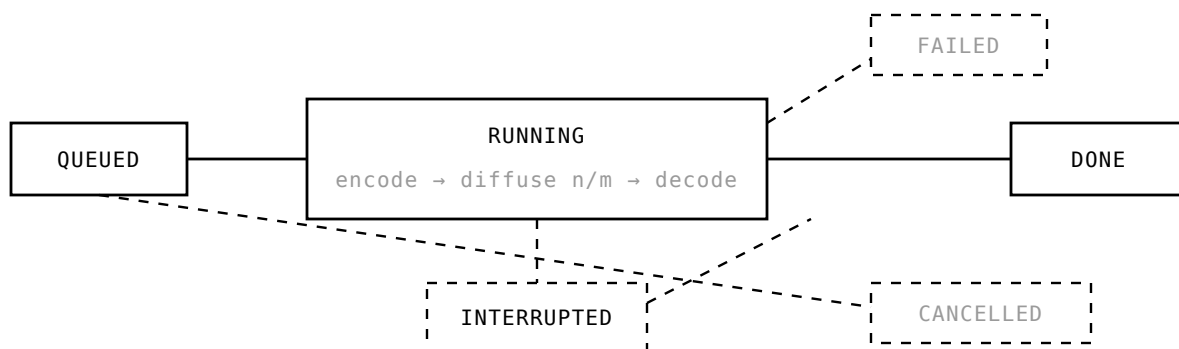
Binary frames

After `result`, the audio arrives as WebSocket *binary* frames, each self-describing so frames from two parallel fetches can't be confused:

```
// binary frame layout – 26-byte header, then payload
[ job_id: 16 bytes (ULID) ][ offset: u64 LE ][ flags: u16 ]( chunk bytes... )
// flags bit 0: last chunk. Chunk size: 64 KiB.
// resume = re-fetch with offset; verify sha256 at the end.
```

PART 4 · THE STATE MACHINE

A job's whole life



Solid – the happy path. Dashed – the honest paths: interrupted (process death; node resumes from the last completed stage), failed (with error code), cancelled (from queued or running). Terminal states keep their `JobView` for `retention_days`.

- `interrupted` is a first-class state, not an error — the node's process died mid-job (battery, reboot). On restart the node re-reads its job store and resumes from the last completed stage; clients just see `interrupted → running`. This matches the stage-resumable design the on-device engine already has.
- Every push is a **full `JobView`**, never a delta — rule 2 applied. Rendering is always "replace what I knew."
- Terminal jobs (`done/failed/cancelled`) persist with their results for `retention_days`, then the node garbage-collects.



Same words, three pipes

TRANSPORT	PIPE	AUTH	USED FOR
in-process	JSI Turbo Module callbacks carrying the same JSON envelopes; binary chunks as <code>ArrayBuffers</code>	none — same device	on-phone GGML, the flagship path
direct	WSS to <code>node:4820</code> ; node key pinned at pairing stands in for a CA	challenge–response	VPS with public IP, laptop on same Wi-Fi
relay	end-to-end encrypted tunnel spliced by cantor-relay; frames inside are byte-identical	challenge–response	everything behind NAT

The app's engine service holds a `Transport` interface with three implementations, and everything above it — stores, screens, progress UI — is written once against the messages. This is the fake-engine seam the app already has, promoted to a contract.

How the server is actually written

The Linux/desktop backend — call it `cantor-node` — is a Rust shell around the same C++ engine the phone uses. Rust because: memory safety at the network edge (this process faces the internet), tokio's async runtime is built for thousands-of-connections work, it compiles to the single static binary the one-command installer wants, and — the quiet superpower — the protocol types can be *generated from it*.

THE LOAD-BEARING BOUNDARY

One C header — `cantor_engine.h` — is the only doorway to the C++/GGML pipeline. The Android JSI module wraps it; the Rust node wraps it. **Two shells, one engine, zero drift.**

```
// cantor_engine.h – the entire FFI surface
ce_ctx* ce_load(const char* model_dir, ce_opts);
int      ce_generate(ce_ctx*, const ce_params*,
                    ce_progress_cb on_progress, // stage, step, total
                    const atomic_bool* cancel,  // checked between steps
                    const char* out_path);
void      ce_free(ce_ctx*);
```

Crate layout

CRATE

CONTENTS

cantor-proto	Every type in Part 3 as serde structs, annotated with <code>ts-rs</code> — <code>cargo test</code> regenerates <code>protocol/*.ts</code> , which the React Native app imports. Rust is the source of truth; the TypeScript cannot drift.
cantor-engine	The unsafe FFI wrapper over <code>cantor_engine.h</code> , exposing one safe async fn: <code>generate(params, progress_tx, cancel) → Result</code> . Runs the engine on a dedicated thread (<code>spawn_blocking</code>); progress flows through a tokio broadcast channel.
cantor-node	The daemon: session handling (axum + tungstenite for direct WSS, an outbound tungstenite client for the relay line), the job manager, allowlist, identity keys (ed25519-dalek), and the job store.

Inside cantor-node

- **Job store = SQLite** (rusqlite): one `jobs` table mirroring JobView, plus a results directory of audio files. Every state transition is written before it's announced — this is what makes `interrupted → running` resume possible after a crash.
- **Job manager**: a queue task feeding `limits.concurrent` workers (default 1 — GGML saturates the machine; a GPU box raises it in config). Each worker drives `cantor-engine`, writes transitions to SQLite, and broadcasts the fresh JobView to every connected session of the job's owner.
- **Sessions are stateless**: a session is just "an authenticated pipe + a subscription to my jobs' broadcasts." Ten phones can watch the same node; a session dying changes nothing about any job. Rule 2, enforced by architecture.

- **Both server and client:** the same session code serves an inbound direct WSS listener and the single outbound relay line — the frames are identical, so the two are one code path behind the `Transport` seam, mirroring the app.
- **Config** is one TOML: `name`, `model_dir`, `relay_url`, `listen`, `allowed_keys` path, `concurrent`, `retention_days`. The installer writes it; the QR/pairing code is printed from it.

PART 7 • TO DISCUSS

Open questions for the taste pass

- **GenParams final list** — I included the obvious ACE-Step knobs (prompt, lyrics, duration, seed, steps, guidance). Does 1.5 expose others worth first-classing (scheduler, language, reference audio later?), or do we add an escape-hatch `extra:map` now?
- **Formats** — master FLAC always, plus an optional Opus "small" render for fast fetch over relays? Or one format per job as drafted?
- **Multi-model** — nodes advertise a model list; is v1 exactly one model per node, keeping `model` required but boring?
- **Result retention default** — 7 days? 30? Phone-nodes may want much shorter.
- **Queue fairness** — when three friends share one VPS: FIFO, or round-robin per pubkey? (FIFO is my draft answer for v1.)
- **Cancellation granularity** — between diffusion steps only (matches the JSI design). Encode/decode stages are short enough to be uncancellable — acceptable?

Why this shape holds: the six principles are the four seams from the network doc wearing work clothes — durable client-named jobs, a versioned envelope, one protocol over

any pipe, keys underneath everything. If we keep faith with those, every phase of the roadmap plugs into this spec without a single breaking change.

EXPLAINED FROM ZERO

How your phone talks to your laptop

The whole network design as one story: why devices can't reach each other, the trick that fixes it, what the keys and QR codes actually do, and why a server in the middle doesn't break "decentralized."

PART 1 · THE PROBLEM

Why your laptop is unreachable

Your laptop *does* have an IP — but it's a private one (something like `192.168.1.34`) handed out by your home router. The router holds the single public IP for your whole house. When your phone, out on LTE, tries to call that public IP, the router receives the call and thinks: *"someone is calling in, but nobody inside asked for this"* — and drops it. That's NAT. Your instinct that every device should have its own reachable IP is exactly right — that's what IPv6 was supposed to fix, twenty years ago. It didn't, so we work around it.

But the workaround exists because of one asymmetry:

THE ONE IDEA EVERYTHING RESTS ON

A device behind a router can't **receive** calls — but it can always **make** calls.

Your laptop can't be called. But it can call out — that's just normal internet use; routers always allow outgoing connections and their replies. Every design decision below is a consequence of this asymmetry.

PART 2 · YOUR IDEA, UPGRADED

From polling to an open line

You proposed: the laptop asks the server every ten seconds, "anything for me to generate?" That **works** — it's a real pattern, called polling, and plenty of systems started that way. There's a standard upgrade:

Instead of calling every ten seconds and hanging up, the laptop makes **one call and never hangs up**. This is a WebSocket: a connection the laptop opens *outward* to the relay and simply keeps open, for hours. Because the laptop initiated it, the router allows it — and now the relay can push messages down that open line the instant they arrive.

POLLING	You phone the office every ten seconds asking "any messages?" — the answer is usually no. Wasteful, and up to ten seconds of delay on every job.
OPEN LINE	You phone the office once and stay on the line . The moment something arrives, they just say it. Zero delay, less battery, less traffic.

And the open line gives you a second thing entirely for free: **presence**. Is the laptop online? Just check whether its line is still open. The moment it shuts down or loses Wi-Fi, the line drops and the relay marks it offline. That is your green dot in the backend picker — no separate ping system to design.

The cousins you've probably met

You're right that WebSockets have siblings, and that the big AI companies use them. The map:

SSE	What ChatGPT streaming actually uses. Server-sent events: a one-way pipe, server → client, over plain HTTP. Perfect when the client asks once and then only listens. Can't carry messages back up.
WEBRTC	What OpenAI's Realtime voice moved to. Built for live audio/video where every millisecond of latency is audible. Heavy machinery: it exists to solve echo, jitter, and packet loss for media — none of which we have.

Ours. Both directions, one long-lived line: submit jobs, cancel, receive progress — all multiplexed on the same connection. The boring, correct tool for a control channel.

So OpenAI didn't leave WebSockets because they're bad — they left for a media-specific tool because voice is media. Our traffic is small control messages plus one file at the end. WebSocket is exactly the conventional choice, including on mobile: React Native has WebSocket built in, and WhatsApp, Slack, Discord, and Firebase all live on persistent sockets from phones.

The ports worry, dissolved

Good news: a server does **not** spend one port per user. It listens on *one* port (say 443), and every connection is identified by four numbers together: *client IP, client port, server IP, server port*. Two users connecting to the same server port differ on the client side, so the server tells them apart without spending anything.

The ~65,000 number you remembered is the port range on *one machine* — it limits how many connections a *single client* can open to the same server, which is never the problem. Servers routinely hold hundreds of thousands of open sockets on one port (WhatsApp famously ran about two million per machine); the real limits are memory and file descriptors, and they're far away at our scale.



One generation, start to finish

The cast:

PHONE	LAPTOP	RELAY
The remote control. Where you type the prompt and watch progress.	The worker. Runs the C++/GGML backend, does the actual generating.	The switchboard. Dumb on purpose. The only machine

		that must have a public IP — that is its whole job.
--	--	--

1 The laptop dials in and stays on the line

The backend starts, calls the relay, and keeps the line open: **"I'm laptop `pk-L...`, here's proof"** (a signature — see Part 4). "I accept jobs from these people: [your key]." Then it sits idle, line open.

2 Your phone dials in and asks who's around

You open the app. The phone calls the relay too: "I'm `pk-C...`, here's proof. Which of my machines are online?" The relay checks its open lines: **"Laptop — online. VPS — online. Home PC — offline."**

3 You hit generate

The phone encrypts the request (prompt, params) **so that only the laptop's key can decrypt it**, and sends it down its line with a note: "deliver to laptop."

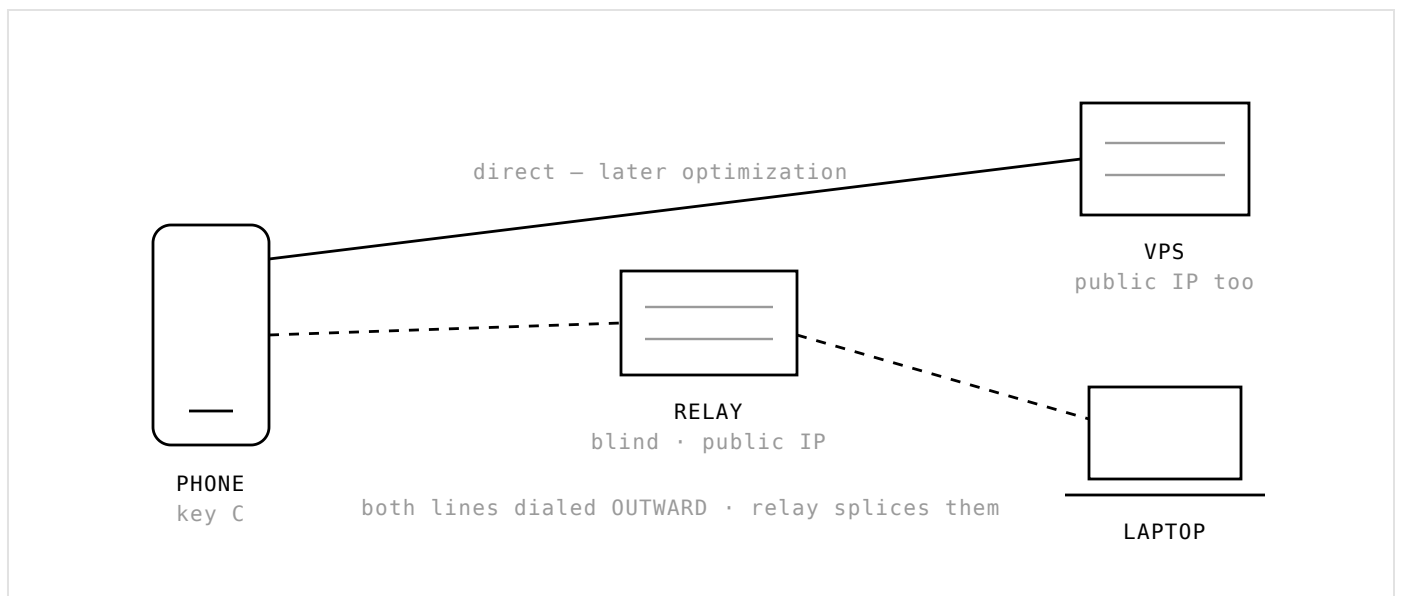
4 The relay copies bytes it cannot read

The relay moves those bytes from your line onto the laptop's line. To the relay it's noise passing between two open calls. **This copying is all "relaying" means.**

5 The laptop works, you watch live

The laptop decrypts, runs the model, and streams progress events back the same way — the step counter on your phone is live. Then the finished song travels the same path home.

That's the entire system. Every machine either *is* reachable (the relay) or *calls out* to it (everyone else). Nobody ever needs to discover anybody's IP.



Dashed – the v1 path: phone and laptop each call out to the relay; the relay splices the two open lines. Solid – direct connection, a later optimization for when a machine (like a VPS) is publicly reachable.



PART 4 · RESILIENCE

When the line breaks — and it will

Connections die constantly on mobile: you step into an elevator, Wi-Fi hands off to LTE, Android puts the app to sleep. None of that may ever lose a song. The design principle that makes breakage boring:

DESIGN PRINCIPLE

The connection is **disposable**. The job is **not**.

Who owns the truth

The **worker owns the job**. When the laptop accepts a generation, it writes a job record to its own disk and walks it through a state machine: `queued → writing lyrics → diffusing 34/57 → decoding audio → done`. The socket dying doesn't touch any of that — generation continues; the job record survives even a reboot.

The phone, meanwhile, never trusts its own picture. Sockets reconnect automatically (retry with growing pauses), and the *first* message after every reconnect is: "**status of my jobs?**" Whatever happened while you were in the elevator, the truth comes back and the UI redraws from it. The screen renders *state*, not a fragile stream of events — miss ten progress messages and nothing is wrong. This is the same philosophy the app already has on-device: process death is normal, stages are resumable.

The app is closed. How does the notification arrive?

Honestly: when you close the app, Android kills its socket — a phone cannot hold an open line all day, and shouldn't. So "your song is ready" arrives by the one channel Android keeps open for everyone: platform push (FCM — Google's push service; every messaging app on Android rides it).

The flow: laptop finishes → tells the relay "job done for `pk-C...`" → the relay sends a **content-free wake-up ping** to your phone's push token. The app wakes in the background, reconnects over its own encrypted line, fetches the real state, and shows the notification itself. The push channel carries no prompt, no lyrics, no audio — just "something changed." Signal works exactly this way, and it's why this doesn't break the privacy story. (For de-Googled phones there's UnifiedPush later; same shape, different postman.)

And when you next open the app mid-generation, it's the same resync move as always — connect, ask, redraw: *diffusing, step 41 of 57*.

The song comes home in pieces

The finished file downloads **chunked and resumable** — like the checkpoint downloads already are. A multi-megabyte song shouldn't restart at byte zero because you walked through a dead spot.



Every device mints an **Ed25519 keypair**: a private key (kept secret, never leaves the device) and a public key (safe to share anywhere — it is your identity, the `pk-C...` in the story above).

"Here's proof" in the story works like this: the relay or a backend sends you a random number, your device signs it with the private key, and anyone holding your public key can verify the signature. **No password, no account, and no secret ever crosses the wire.**

SO WHAT WERE THE SIXTEEN WORDS?

Nothing but a **paper backup of the private key**. A private key is 128 random bits — impossible to write down reliably — so the wordlist encodes those same bits as words a human can copy onto paper. Same number, two costumes. The app can generate the keypair directly and never show words at all, offering "back up your identity" later. Words are typed in exactly one situation: restoring your identity on a new personal phone. Never into a server. Never for pairing.

Pairing: the QR code (or a short code)

What pairing must solve: *two devices need to learn each other's public keys, without trusting any server to introduce them.*

When the laptop installs the backend, it prints a QR containing its **public key** and **which relay it listens on**. You scan it — now your phone knows how to find and verify the laptop — and your phone sends its own public key over, which the laptop adds to its `allowed_keys` list. Paired, permanently. The exchange traveled through your eyeballs and camera, which nobody can intercept.

A temporal code (like `7QK4-BM2N`) is the same content in a typeable costume, for when there's no camera to point — a VPS you only reach over SSH, say. Support both.

Sharing a server with friends is the same move again: their public keys become three more lines in `allowed_keys`.

A server in the middle — is this still decentralized?

Yes — and here's the honest definition. Decentralized doesn't mean *no servers*. It means **no mandatory, trusted, single owner**. Check the relay against all three:

IDENTITY ISN'T OURS	THE RELAY IS BLIND	THE RELAY IS REPLACEABLE
Your key was minted on your phone. Not a row in our database. We can't ban it, lose it, or own it.	It moves ciphertext between open lines. Whoever runs it learns nothing worth having — no prompts, no lyrics, no audio.	An open-source program anyone can run. The app lets you point at any relay — ours is a default, not a requirement.

Like email: Gmail's servers exist and your own mail server exists, and they're equally valid citizens. Nostr, Anytype, and Syncthing all have this exact shape — servers exist, but they're dumb, blind, and swappable. Compare the centralized version:

	SPOTIFY	CANTOR
Identity	Their account, their database	A key you minted yourself
Servers	Theirs only, mandatory	Blind relays, anyone can run one
Content	Readable by them	Encrypted end to end — relay sees noise

Every flipped cell is what "decentralized" means here. "Peer-to-peer" is the aspiration; *this* table is the substance.

PART 7 • HOSTING

Where the relay lives (yes, Workers can do this)

Your Cloudflare instinct is good. Workers support WebSockets, and the right shape there is a **Durable Object**: a tiny stateful mini-server Cloudflare spins up on demand that can hold many open WebSocket lines at once, remembers things (our who's-online table, the mailbox), and — the killer feature — *hibernates*. Between messages the object sleeps and costs nothing, while its WebSocket lines stay connected. An idle relay is nearly free.

And Durable Objects are on the **free plan** now (SQLite-backed, 1 GB storage, ~100k requests/day). At early scale the relay costs \$0 and there's no VM to babysit.

There's even an elegant self-hosting twist: because the relay is a Worker, **anyone can deploy their own copy to their own free Cloudflare account with one command** (`wrangler deploy`). "Run your own relay" on day one, without asking anyone to rent a VM.

The honest trade-offs, so nothing surprises us later:

- **Song bytes flow through Cloudflare.** Still ciphertext — the blindness holds — but heavy audio relaying will eventually cost real money on any host. That's precisely what the direct-paths phase exists to relieve.
- **Cloudflare-only would be a soft centralization.** The guard: keep the protocol boring — WebSocket plus framed messages — so a native Go/Rust binary on any \$4 VM (or Oracle's free tier) can speak it identically later. *The protocol is the standard; hosts are interchangeable.*

So the recommendation: **v1 relay = one Worker + Durable Objects on the free tier.** Native binary later, when someone actually wants a non-Cloudflare home.



PART 8 • SHARING MUSIC

Centralize the index, not the identity

Two different kinds of sharing, two different answers.

To a friend: fully decentralized, same rails

A song becomes a file plus metadata **signed by your key** — authorship is provable ("this came from `pk-C...`") and unforgeable. Sending it to a friend is your phone reaching *their* device exactly the way it reaches your laptop: through a relay, encrypted to *their* public key. If they're offline, the relay's mailbox holds the sealed package until they connect. Nothing new to invent.

To the world: your instinct is right — and it doesn't break the philosophy

A public feed — discovery, rankings, search, streaming on demand — needs something always-online, fast, and curated. Fighting that with pure decentralization is how you get a product nobody can actually browse. So yes: run a **centralized index**. The trick that keeps the soul intact is *what* gets centralized:

- The index stores and serves **songs that are signed by their creators' keys**. It can host, rank, and recommend — it can never forge authorship or claim your catalog, because it doesn't own any identity.
- Your identity, your followers-of-keys, your catalog are **portable**. If our feed dies or turns hostile, another index — anyone's — can serve the same signed content, and you lose nothing. (This is Bluesky's shape: convenient center, exitable at will.)
- Friend-to-friend sharing keeps working with no index at all.

Centralized *service*, decentralized *ownership*. Build it long after the generation network works.

PART 9 · BUILDING IT RIGHT

The four seams that make it patch-free

"Design it properly from the beginning" doesn't mean building everything now — it means placing the seams so that everything later slots in without surgery. There are exactly four:

1 · DURABLE JOBS Every generation has an ID and a worker-owned state machine on disk. Connections carry views of jobs, never the jobs themselves.	2 · VERSIONED MESSAGES Every message carries a protocol version field. Old phones and new workers can always negotiate instead of breaking.	3 · ONE PROTOCOL, ANY TRANSPORT The same framed messages travel in-process, over LAN, or through a relay. Swapping WebSockets for anything else later touches one layer.
4 · KEYS FROM DAY ONE Identity is the Ed25519 keypair before any networking ships. Signatures on jobs and songs cost		

nothing now and unlock
everything later.

Everything else — hosting, hole punching, feeds, even the relay itself — is allowed to change, because these four seams hold still.



PART 10 • THE PLAN

Five phases, each one shippable

One clarification first: **Go** was nothing conceptual — just a suggested programming language for a native relay binary. With the Workers plan above, the v1 relay is TypeScript on Cloudflare instead, and a Go/Rust binary becomes the later, VM-friendly sibling. The app stays React Native; the backend stays C++/GGML; the relay is a separate little program either way.

0 One protocol, everywhere

Define the backend conversation once — `submit(prompt, params) → job_id`, progress events, `cancel`, `fetch_result` — and make on-phone GGML, desktop, and remote servers all speak it. The app keeps exactly one code path; only the transport changes: in-process, LAN, or relay.

SHIPS: THE INTERFACE EVERYTHING ELSE PLUGS INTO

1 Direct remote, no infra

One-command Linux installer: downloads checkpoints, mints the server's keypair, writes `allowed_keys`, listens on a port. Pair by QR or short code. Works today for VPSes (public IP — directly reachable, no relay needed) and laptops on your own Wi-Fi.

SHIPS: GENERATE ON YOUR VPS FROM YOUR PHONE

2 cantor-relay — the switchboard

The small server from the story: key-authenticated open lines, a who's-online table, splicing encrypted streams, holding mailbox packages. Built as a Worker + Durable Objects on the free tier; self-hostable via `wrangler deploy`. FCM wake-up pings for finished jobs land here too. Open-sourced; the app accepts any relay URL, defaulting to `relay.cantor.app`.

SHIPS: WORKS BEHIND ANY ROUTER · LIVE GREEN DOT · DONE-NOTIFICATIONS

3 Direct paths, relay as fallback

In v1 every song's bytes flow through the relay — blind, but bandwidth we pay for. This phase makes devices attempt a direct connection when possible (VPS, same Wi-Fi, hole punching), falling back to relay. This is where a library like **Iroh** earns its keep: it dials peers by public key and handles direct/punched/relayed automatically.

SHIPS: RELAY COSTS DROP TO MATCHMAKING ONLY

4 Social layer

Names without accounts: profiles signed by your key (display name, avatar) plus local petnames ("Dad's PC," "Ana") — your address book, nobody else's. Share-a-server in one scan. Song sharing to friends via the mailbox. Public feeds only after all this works.

SHIPS: LEND YOUR GPU TO A FRIEND IN ONE SCAN

Do now, while it's cheap: make identity be the Ed25519 keypair itself — generated directly, stored in the Keystore — with the word-phrase demoted to an optional "back up your identity" feature. Everything above hangs off that one decision, and it's the only part that gets expensive to retrofit.

Implementation notes

Phase 1 — Presence + splice

Started: 2026-07-20

Scope

- Complete the relay-level frame vocabulary.
- Verify Ed25519 room claims.
- Broadcast node presence to attached clients.
- Splice opaque tunnel payloads through relay-assigned session IDs.
- Make the most recently authenticated node connection own the room.
- Preserve hibernation-safe per-socket state and automatic ping/pong handling.
- Cover claim, presence, splice, reconnect, and bad-claim behavior in the Workers runtime.
- Add a small command-line client for the local presence demo.

Implementation log

- Confirmed Phase 0 already provides the Worker route, SQLite-backed `NodeRoom`, hibernating node sockets, Ed25519 claim verification, and the Rust node's local claim loop.
- Confirmed the target protocol keeps application payloads opaque to the relay; only the relay envelope and session ID are inspected.
- Added client socket attachments with relay-generated UUID session IDs and hibernation-safe `client / session:<sid>` tags. The relay uses runtime socket discovery rather than an in-memory connection map.
- Added `relay.presence` and `tunnel` envelopes, client-to-node forwarding with an injected session ID, node-to-client routing with the session ID removed, and non-terminal `node-offline / client-offline` errors.
- Completed Ed25519 claim validation with encoding, length, import, and signature failures all mapped to `bad-claim`.
- Implemented reconnect-wins by authenticating the newest node, closing any authenticated predecessor with code 1012, and preserving truthful online presence while the predecessor's close event drains.
- Kept the Hibernation API and application-level automatic `ping / pong` pair. Per-socket attachments are the only connection state required after eviction.
- Explicitly completes WebSocket close handshakes in `websocketClose`. Current Cloudflare runtimes also auto-reply, and the explicit call is documented as safe; it makes local Workerd cleanup deterministic.
- Added the current Workers test stack: `@cloudflare/vitest-pool-workers` 0.18.6 with Vitest 4.1.10. The test TypeScript project skips third-party declaration checking because the pool's bundled declarations conflict with the latest standalone Workers declarations; all project and test source remains strictly checked.
- Added `relay/scripts/client.mjs` and documented the local demo in `relay/README.md`.

Validation

- `npm test` in `relay/`: 4 tests passed (presence transitions and offline behavior, hibernation + ping/pong + bidirectional splice, reconnect-wins, and bad Ed25519 claim).
- `npm run check` in `relay/`: generated bindings current; Worker and test TypeScript checks passed.
- `npm run deploy:dry-run` in `relay/`: passed with the `R00MS` Durable Object binding recognized.
- `cargo test --workspace` in `node/`: 3 tests passed.
- Live local demo: the command-line client observed `presence: online`, then `presence: offline` after stopping the Rust node, then `presence: online` after restarting the same identity.
- Physical Android device `6b1f6ba8629c`: `com.cantor.app` remained the focused, resumed activity (PID 11180), with no recent fatal Android/React Native errors.
- `git diff --check`: passed.

Phase 2 — Protocol + node handshake

Started: 2026-07-20

Scope

- Add a shared Rust protocol crate and committed generated TypeScript types.
- Complete the end-to-end client/node challenge-response handshake.
- Enforce the client-key allowlist and the single-use pairing enrollment path.
- Return static node capabilities and an empty job status snapshot.
- Add `pair` terminal output and a reconnecting node relay loop.
- Exercise accepted and rejected clients against the local relay.

Implementation log

- Added the `cantor-proto` workspace crate with the flat versioned `hello / challenge / auth / welcome / status / jobs / error` vocabulary, minimal `NodeInfo`, and `JobView`. `ts-rs` tests regenerate the committed TypeScript definitions under `protocol/`.
- Implemented an independent handshake state machine per relay-assigned session. It validates base58 Ed25519 keys, binds `auth` to the request and 32-byte challenge, verifies the signature, and rejects status requests until authentication succeeds.
- Added allowlist enforcement and atomic owner-only config replacement for one-time enrollment. Invalid signatures cannot consume a pairing token or alter the allowlist.
- Added `cantor-node pair`, terminal Unicode QR output, and a copyable percent-encoded pairing URI. `run` accepts only previously paired keys.
- Added static Phase 2 capability data and the empty `jobs` status response; engine execution remains deliberately stubbed.
- Reworked the node relay connection into a persistent loop with a 1–30 second capped exponential delay and up to 250 ms of operating-system-random jitter. A successful claim resets the retry attempt.
- Added `node/scripts/protocol-client.mjs` for the complete handshake/status demo and documented pairing, allowlist, and binding generation behavior.

Validation

- `cargo test --workspace` in `node/`: 15 tests passed, including generated TypeScript exports, wire shape, pairing enrollment, rejection, atomic config persistence, QR fields, and reconnect-delay bounds.
- `cargo clippy --workspace --all-targets -- -D warnings`: passed.
- Live local relay/client: a new Ed25519 client enrolled with the one-time token, received `welcome` with the static `NodeInfo`, requested `status`, and received `jobs: []`.
- Repeated live client with the persisted identity and no token: accepted.
- Fresh live client with no token: received application error `rejected` and did not enter the allowlist.
- Relay restart demo: the running node observed the disconnect, retried with jittered 1/2/4/8/16-second backoff, and reclaimed the same room once the local relay returned.

- `git diff --check:passed.`

Phase 3 — App identity + first backend

Started: 2026-07-20

Scope

- Derive a stable Ed25519 app identity from the onboarding mnemonic and protect its secret with Android Keystore.
- Persist paired backend records locally.
- Add QR scanning with a clipboard fallback.
- Implement the app connection lifecycle, signed handshake, reconnect, and status refresh.
- Render the paired node's capability card and exercise the flow on the physical Android device over the LAN relay.

Implementation log

- Derived a domain-separated 32-byte Ed25519 secret from the BIP39 seed with HKDF-SHA256 (`cantor-identity-v1`), encoded the public key as base58, and added challenge signing with Noble's primitives. The temporary BIP39 seed buffer is cleared after derivation.
- Stored the derived secret with `react-native-keychain` under the app-specific service using AES-GCM Keystore storage and a secure-software minimum. App boot now loads that identity before deciding whether onboarding is required.
- Added validated AsyncStorage persistence for the exact backend record shape: node public key, relay URL, petname, and last known `NodeInfo`.
- Added the Android camera permission, VisionCamera's built-in QR code scanner, an explicit camera-permission action, and the planned clipboard fallback.
- Added strict pairing URI validation, including base58 public-key and base64url token lengths, safe WebSocket relay normalization, and rejection of custom-scheme lookalikes.
- Implemented the app relay lifecycle (`disconnected` , `connecting` , `attached` , `handshaking` , `ready`), node-key verification, signed challenge response, one-time enrollment, status request, validated capability/job parsing, and jittered reconnect capped at 30 seconds.
- Added the Backends screen with persisted capability facts, live connection labels, zero-job/load status, and a green ready state. Added native-module Jest mocks plus identity and pairing parser coverage.

Validation

- `npm test -- --runInBand` in `cantor/`: 118 tests passed across 7 suites.
- `npx tsc --noEmit` in `cantor/`: passed.
- `npm run lint` in `cantor/`: zero errors; the single inline-style warning in `src/onboarding/panels/kit.tsx` predates Phase 3.
- `./gradlew app:assembleDebug` in `cantor/android/`: passed, including native Keychain, AsyncStorage, and VisionCamera compilation with QR scanning enabled.

- Installed the debug APK on physical Android device `6b1f6ba8629c`. Completed onboarding once, then restarted the app and confirmed it opened directly to Backends with the same app key (`AaEmzBWq...bgoZpE`).
- Granted camera access through the in-app action and confirmed a live rear camera preview and QR detection on the device.
- Scanned the live LAN pairing QR. The node authenticated and persisted the app key; the app reached `READY` and rendered `studio-linux`, `linux-x86_64`, `ace-step-1.5-stub`, `ace-step-1.5`, advertised limits, zero load, and zero jobs.
- Stopped the node and observed `NODE OFFLINE` while the last capability data remained visible; restarted it without a pairing token and observed an automatic return to `READY`.
- Force-stopped and relaunched the app after pairing. The stored identity and backend loaded without onboarding and automatically reconnected to `READY`.
- Recent device logs contained no fatal Android or React Native errors.
- `git diff --check`: passed.

Phase 4 — Deploy + real-world test

Started: 2026-07-20

Scope

- Deploy the tested relay and Durable Object migration to Cloudflare.
- Attach the relay to `cantor.ckadirt.xyz` as a Worker Custom Domain.
- Add a Linux node installer stub covering the binary, private config, and a systemd service.
- Validate the production `wss://` path with the node behind the home router and the physical phone on LTE, with no inbound port forwarding.

Implementation log

- Confirmed Wrangler 4.112.0 is installed and authenticated with Worker, Durable Object route, certificate, and zone access.
- Confirmed `relay/wrangler.jsonc` already declares the current compatibility date, SQLite `NodeRoom` migration, observability, and the exact `cantor.ckadirt.xyz` Custom Domain. The hostname has no pre-existing DNS record, so Wrangler can create the originless Custom Domain record and certificate without replacing user DNS.
- Deployed Worker version `9e11285c-54ed-45ed-aa0a-31872e12b5b5`; the Custom Domain now resolves on public DNS and serves the expected Worker responses over a valid TLS certificate.
- Added the non-root Linux installer stub with checksum-verified release downloads, owner-only config, symlink/unmanaged-unit refusal, and a hardened per-user systemd service that remains stopped until pairing is complete.
- The first production `wss://` attempt exposed that `tokio-tungstenite` 0.30's Rustls feature does not select a process crypto provider. Added an explicit Ring provider installation at process startup so secure WebSockets cannot reach Rustls's ambiguous-provider panic.
- Paired the existing physical app with the production relay. The deployed path reached `READY`, reported `NODE OFFLINE` when the production-connected node stopped, and returned automatically to `READY` when normal node run mode restarted.
- Cold-launched the app after restoring the phone's original DNS settings. Its persisted identity and production backend reconnected to `READY` without onboarding or another pairing scan.
- A final installer test found that line-oriented `grep` does not treat an embedded newline as a matched control character. Added an explicit newline guard before the general control-character check.
- Phase 4 implementation is complete, subject to the documented no-SIM cross-network acceptance limitation.

Validation

- `npm test` in `relay/`: 4 tests passed.
- `npm run check`, `npm run deploy:dry-run`, and `npx wrangler check startup` in `relay/`: passed.

- Production deployment: Worker version `9e11285c-54ed-45ed-aa0a-31872e12b5b5` is attached to `cantor.ckadirt.xyz`; Cloudflare and Google public resolvers return the Custom Domain addresses, HTTPS presents a valid certificate, `/` returns the expected JSON 404, and a room route without an upgrade returns the expected JSON 426.
- `cargo test --workspace` in `node/`: 16 tests passed, including explicit TLS provider installation.
- `cargo clippy --workspace --all-targets -- -D warnings` and `cargo build --workspace --release` in `node/`: passed.
- Installer: shell syntax, local-binary installation, managed reinstall with config preservation, owner-only config modes, executable/service modes, installed binary startup, `systemd-analyze --user verify`, newline/control rejection, symlink refusal, and unmanaged-unit refusal passed. ShellCheck is not installed in this environment.
- Physical Android device `6b1f6ba8629c`: paired through `wss://cantor.ckadirt.xyz` and reached `READY`; stopping the node produced `NODE OFFLINE`; restarting it in normal `run` mode restored `READY`; a cold app relaunch under the phone's restored default DNS settings also reached `READY`.
- `npm test -- --runInBand` in `cantor/`: 118 tests passed across 7 suites.
- `npx tsc --noEmit` in `cantor/`: passed.
- `npm run lint` in `cantor/`: zero errors and the same pre-existing inline style warning in `src/onboarding/panels/kit.tsx`.
- `git diff --check`: passed.

Post-Phase 4 — High-priority hardening

Started: 2026-07-20

Scope

- Keep the one-time pairing secret out of relay-visible application frames.
- Reclaim per-client node handshake state on disconnect and cap retained state.
- Prevent transient or permanent secure-storage read failures from opening the identity-creation path.
- Leave generation-specific protocol and transport work for the later generation phase requested by the user.

Implementation log

- Replaced the plaintext `pair_token` hello field with `pair_proof`, computed as HMAC-SHA256 over a domain separator plus the raw node and client public keys. The app and command-line client derive the proof locally from the QR token; the node verifies it in constant time and still consumes the token only after a valid Ed25519 challenge signature and atomic allowlist persistence.
- Added one cross-language golden proof vector to the Rust and TypeScript test suites so encoding, domain separation, and key order cannot silently drift.
- Added the relay-only `relay.detached` frame. A client close/error notifies the currently authenticated node, which idempotently removes that session's handshake/authentication state.
- Added a hard ceiling of 1,024 simultaneous client sessions per node relay connection. Existing sessions continue at the ceiling; a new session gets a versioned `too-many-sessions` application error without allocating state.
- Changed app identity boot failures to a dedicated `IDENTITY LOCKED` screen with an explicit retry action. Onboarding is never rendered after a Keychain read error, so the stable identity cannot be overwritten through that error path.
- Updated the node and relay operator documentation for the proof and detach behavior. The QR format remains unchanged and the raw one-time token remains local to the QR holder and node process.

Validation

- `cargo test --workspace` in `node/`: 18 tests passed, including pairing-proof binding and session-limit behavior; generated TypeScript bindings are current.
- `cargo clippy --workspace --all-targets -- -D warnings`: passed.
- `cargo build --workspace --release` in `node/`: passed.
- `npm test` in `relay/`: 5 tests passed, including detach notification.
- `npm test -- --runInBand` in `cantor/`: 120 tests passed across 7 suites, including pairing proof and non-destructive identity retry coverage.
- `npm run check` in `relay/`, `npx tsc --noEmit` in `cantor/`, and `node --check node/scripts/protocol-client.mjs`: passed.
- `npm run deploy:dry-run` and `npx wrangler check startup` in `relay/`: passed; no production deployment was performed.

- Reviewed the complete Worker against current Cloudflare Workers and Durable Objects guidance plus the latest published Workers type definitions. The hibernation attachments, close/error handlers, generated `Env`, current compatibility date, SQLite migration, and observability configuration match the current APIs and recommendations.
- Live local end-to-end pairing: the command-line client sent the new proof, the node persisted its verified Ed25519 key, returned `welcome`, and returned `jobs: []`; the relay observed the normal client detach afterward.
- `./gradlew app:assembleDebug` in `cantor/android/`: passed.
- Physical Android device `6b1f6ba8629c`: after wake and cold launch, the app loaded the same saved app key directly into Backends with no onboarding or fatal React Native/Android errors. The card correctly showed `NODE OFFLINE` because no production node process was running. The protected identity was not modified to induce the tested failure path.
- `npm run lint` in `cantor/`: zero errors and the same pre-existing inline style warning in `src/onboarding/panels/kit.tsx`.
- `git diff --check`: passed.

Production hardening redeploy + stakeholder packet

Started: 2026-07-20

Deployment log

- Re-ran the relay's Workers-runtime tests, generated-type checks, Worker and test TypeScript checks, and deployment dry run after commit `68ff3b8`.
- Deployed the hardened relay to the existing `cantor.ckadirt.xyz` Custom Domain. Cloudflare assigned Worker version `db43626e-1ab1-4382-8f4b-85dcc643d861` with a reported 5 ms startup time.
- Confirmed through Wrangler deployment status that the new version receives 100% of production traffic.
- Confirmed the public HTTPS endpoint returns the expected JSON `404` at `/` and JSON `426` for a valid room route without a WebSocket upgrade.
- Connected the updated release node using the existing `studio-linux` identity/config and observed `relay.ok`, proving its production room could be reclaimed after the deployment. After the user noted the expected offline state from that stop, restarted the node and cold-relaunched the app to force a fresh relay socket; the physical device returned to `READY`. The node was left running at the user's request.

Stakeholder packet

- The Slack-ready packet combines a current executive/prototype overview, the authoritative first-connection plan, the earlier backend protocol and network rationale as supporting references, and these complete implementation notes.
- The packet states the source precedence explicitly: the locked final plan is authoritative where the earlier guides differ, and generation work remains a later phase.
- Rendered the source bundle as one 47-page, approximately 0.8 MB PDF. Verified the three-page decision brief visually, checked the first page of every source section, and confirmed the merged document remained text-extractable through its final paragraphs.

Validation

- `npm test` in `relay/`: 5 tests passed.
- `npm run check` and `npm run deploy:dry-run` in `relay/`: passed.
- `npx wrangler deployments status`: production is 100% on version `db43626e-1ab1-4382-8f4b-85dcc643d861`.
- Production `studio-linux` node claim: passed.
- Physical Android app cold-reconnect through the redeployed relay: `READY`.
- Stakeholder packet page count, visual samples, and full-text extraction: passed.

Deviations

- During Phases 1–4, the plan’s linked protocol artifact was inaccessible from this environment, so Phase 2 used the smallest versioned `NodeInfo` and `JobView` shapes needed by the final written plan. The user later supplied local copies of the protocol and network guides at the repository root. The final phased implementation plan remains authoritative where they differ; the earlier guides now provide referenced detail where it does not conflict.
- The supplied guides describe generation data-plane records and transports, but the final plan and the user’s explicit sequencing put generation after first connection. This hardening pass therefore leaves those additions for the generation phase instead of expanding the current protocol speculatively.
- The documented pairing URI carries only the node identity and relay address, but the node is also required to reject clients outside its allowlist. To close that authorization gap conservatively, `cantor-node pair` adds a single-use random token to the QR URI. A client proves possession of its key during the normal signed handshake and presents a domain-separated, key-bound HMAC proof of the token; the node then atomically adds the verified client key to its allowlist and invalidates the token. The relay remains unaware of the raw token and cannot reuse the observed proof with a different client key.
- Hermes on the physical Android device does not expose custom `cantor://` hosts or `ws://` URLs consistently through `URL`. Phase 3 therefore validates both prefixes explicitly, parses only their remaining standard components through HTTP(S) sentinel URLs, and reconstructs the WebSocket URL. Lookalike schemes and hosts are rejected before query parsing.
- Requesting camera permission as soon as the pairing modal mounted raced the MIUI activity lifecycle and returned `NO_ACTIVITY`. Phase 3 conservatively requests permission only after the user presses `Allow camera`; the native permission dialog and camera preview were verified on the device.
- Phase 4 uses a per-user systemd service instead of a root-owned system unit. This keeps installation non-privileged and confines the node identity/config to the owning user; the service is intentionally not enabled until the app has completed its first pairing.
- The home ISP resolver retained the pre-deploy NXDOMAIN response after public resolvers had the new Custom Domain. The initial node activation used an unprivileged, process-local hosts overlay pinned to a current Cloudflare edge address. The router later returned the public records, and the final phone cold-launch passed with its original DNS settings restored; the Linux host’s stub resolver still held its own stale negative cache during the test.
- The physical phone has no SIM card, so disabling Wi-Fi removes its only internet path. The attempted Wi-Fi-off check correctly became disconnected but cannot serve as the plan’s LTE acceptance test. Conservatively, Phase 4 verifies both peers using outbound-only production WSS over home Wi-Fi, with no inbound port forwarding, and records the literal cross-network/LTE test as not run rather than claiming it passed. The phone’s Wi-Fi and original Private DNS settings were restored afterward.